Computational Paper and Pop-ups

Nina De La Torre

1 Introduction

Pop-up books contain 3D structures that move as the book opens. While they rely on folding techniques similar to origami, their strength lies in creating collapsible structures driven by a single opening motion. In this project, I extend the mass-spring simulator from *Fast, Interactive Origami Simulation using GPU Computation* (Ghassaei, Demaine, Gershenfeld) to model pop-up cards. More broadly, I extend the simulator to create digital twins of a wider range of paper crafts. This report focuses on four extensions I implemented: The ability to "glue" paper together, the ability to model pre-creased edges, collision handling, and an interactive crease pattern editor.

In order to keep this report brief, I will only give a very broad overview of the existing simulator. The original simulator uses a mass-spring model built with GPU-accelerated WebGL for browser-based interaction. A 2D crease pattern is parsed into nodes (points), beams (edges), and faces (triangles), and there are three main constraints to enforce paper-like behavior. Beam constraints apply Hooke's law to treat edges as springs and preserve rest lengths, face constraints maintain triangle shape over time, and crease constraints drive folds toward a user-specified target angle θ_{target}. Origami Simulator allows you to model mountain folds (blue) valley folds (red), facet edges (yellow), hinges (magenta), cuts (green), and borders (black).

2 Glue Dots

To model pop-up cards, we must connect separate pieces of paper. I introduce "glue dots," specified in an SVG crease pattern as matching pairs of cyan circles. Each pair defines a glue spring: a beam with zero rest length that is not part of any face. This enables attaching multiple sheets and supports adding up to 255 glue connections.

3 Pre-Creased Edges

Certain pop-up mechanisms rely on pre-creased folds that preferentially bend in one direction. Standard hinge-like creases can fold in either direction, which can allow incorrect configurations (see Figures 1 & 2). To address this, I introduce pre-creased mountain and valley edges which are modeled as angular half-springs. These behave as freely swinging hinges in one angular direction, but resist motion in the opposite angular direction.

The crease force is defined as:

$$F_{\text{crease}} = \begin{cases} k_{\text{fold}}(\theta - \theta_{\text{target}}), & \text{if } (\theta - \theta_{\text{target}})s < 0 \\ 0, & \text{otherwise} \end{cases} \qquad \theta_{\text{target}} = \begin{cases} 5, & \text{valley} \\ -5, & \text{mountain} \end{cases}$$

and $s = \text{sign}(\theta_{\text{target}})$. Here, θ is the current fold angle and k_{fold} is the stiffness constant. In the crease pattern, pre-creased mountain edges are pink (rgb(255,0,155)) and pre-creased valley edges are purple (rgb(155,0,255)).

4 Collision Detection and Reaction

One of the biggest challenges in creating pop-up cards is getting it to shut flat. Sometimes you create a beautiful paper card but then realize than it cannot fold into a flat state without introducing new folds or tears. Some pop-up creations also depend on paper collision in their design. In the code, I detect collisions on the CPU then carefully pack the necessary information and send it to the GPU, then in a GLSL shader called collisionVelocityCalc, I add the appropriate forces. On the CPU, I do two checks: node-face collision checks and edge-edge collision checks. The math behind collision detection checks come from the papers *Robust Treatment of Collisions, Contact and Friction for Cloth Animation* (Bridson, Fedwik, and Anderson) and *Incremental Potential Contact: Intersection- and Inversion-free, Large-Deformation Dynamics* (Li et al.).

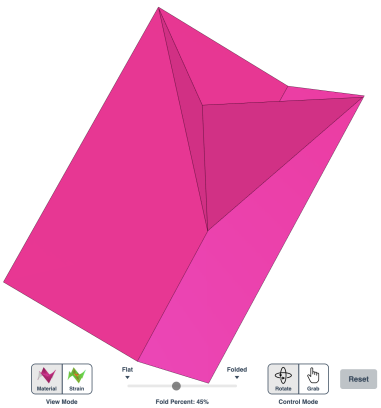

Figure 1: V-Fold using a pre-creased mountain edge in center

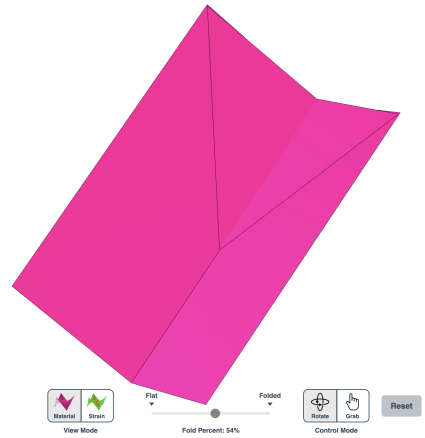

Figure 2: V-Fold using a freely swinging hinge in center

4.1 Node-Face Collision Check

Let triangle x_{123} have vertices x_1, x_2, x_3, and define $x_{ij} = x_i - x_j$. The distance from node x_4 to the triangle plane is:

$$d = |x_{43} \cdot \hat{n}|$$

where $\hat{n}$ is the unit normal that defines the plane containing x_{123}. If $d \leq d_{\max}$, the point is projected onto the plane and expressed in barycentric coordinates (u, v, w) by solving:

$$\begin{bmatrix} x_{13} \cdot x_{13} & x_{13} \cdot x_{23} \\ x_{13} \cdot x_{23} & x_{23} \cdot x_{23} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} x_{13} \cdot x_{43} \\ x_{23} \cdot x_{43} \end{bmatrix}$$

with $w = 1 - u - v$. A collision occurs if $u, v, w \in [0, 1]$ and $d \leq d_{\max}$.

Forces are applied to respective nodes as:

$$F_i = u_i \tilde{I} \hat{n}, \quad i = 1, 2, 3, \quad F_4 = -\tilde{I} \hat{n}$$

where $\tilde{I}$ is the adjusted impulse and $b(d, d_{\max})$ is a smoothly clamped C^2 barrier function:

$$\tilde{I} = \frac{2b(d, d_{\max})}{a^2 + (1-a)^2 + b^2 + (1-b)^2} \qquad b(d, d_{\max}) = \begin{cases} -(d - d_{\max})^2 \ln\left(\frac{d}{d_{\max}}\right), & 0 < d < d_{\max} \\ 0, & d \geq d_{\max} \end{cases}$$

4.2 Edge-Edge Collision Check

For edges $x_1 x_2$ and $x_3 x_4$, the unique closest pair of points on the line segment containing the edges are found by solving:

$$\begin{bmatrix} x_{21} \cdot x_{21} & -x_{21} \cdot x_{43} \\ -x_{21} \cdot x_{43} & x_{43} \cdot x_{43} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} x_{21} \cdot x_{31} \\ -x_{43} \cdot x_{31} \end{bmatrix}$$

where the points are defined from the a, b weights as

$$P_a = x_1 + a x_{21}, \quad P_b = x_3 + b x_{43}.$$

We clamp $a, b \in [0, 1]$ to give the pair of points that exist on our finite edge line segment. A collision is detected if $\|P_a - P_b\| < d_{\max}$. We define the response direction $\hat{n} = \widehat{P_a - P_b}$.

Forces are distributed to respective nodes as:

$$F_1 = (1-a)\tilde{I}\hat{n}, \quad F_2 = a\tilde{I}\hat{n}, \quad F_3 = -(1-b)\tilde{I}\hat{n}, \quad F_4 = -b\tilde{I}\hat{n}$$

5 Interactive Crease Pattern Editor

I implemented an interactive editing mode for modifying crease patterns in real time. Users can select and move nodes directly in the 2D pattern view, and the simulator updates connected edge rest lengths dynamically. This enables immediate visualization of the resulting 3D structure.

6 Limitations and Repository Overview

Collision handling needs work. There are still many cases of self-penetration, and edge-edge collision reactions have a directional problem where if two edges collide they get stuck overlapping. There is a bug in edge-edge reaction forces that can cause the simulation to float away. Future work includes spatial acceleration structures (e.g., AABB) to improve performance and robustness, as well as remeshing the paper to connect glue dots, and support of curved creases.

Glue springs are implemented in `pattern.js`, primarily in `addGlueSprings` and `getCircleParams`. These functions detect pairs of cyan circles (matched via color encoding), identify their associated nodes, and add zero-rest-length beams between them. The codebase also includes support for "glue tabs" (cyan edges acting as hinges with glue at endpoints), though this is redundant and could be removed.

Pre-creased edges are identified in `pattern.js` via `TypeForStroke`, while their behavior is implemented in the `velocityCalcShader` GLSL code in `index.html` (search `isHalfCrease` to find it).

Collision detection is performed on the CPU in `dynamicSolver.js`, mainly in `getNodeFaceCollisionMeta` and `getEdgeEdgeCollisionMeta`, which compute and pack collision data for the GPU. Collision response is handled in the `collisionVelocityCalcShader` GLSL code in `index.html`. Because detection happens only at render steps (not every timestep), the shader rechecks collision conditions before applying forces.

The crease pattern editor is implemented in `pattern.js`, mainly inside `updateHoverCircleForVertex`, `updateConnectedEdge` `updateModelForMovedVertex`, `dragVertexToMouse`, `updateHoveredVertex`, and `setPatternEditingMode`.